

Hand Gesture Virtual Mouse System

Touchless Human-Computer Interaction Using Computer Vision & Artificial Intelligence

Bachelor of Technology

Computer Science & Engineering (Industrial Internet of Things)

3rd Year — 6th Semester

Submitted By

Kavya (Group Leader)
Aarushi Pandey
Riya Dwivedi
Shobhit Saxena
Tanmay Kapoor
Vansh Chitransh
Vivek
Priyanshu Bora
Divya
Mayank

Department of Computer Science & Engineering

Invertis University

Bareilly, Uttar Pradesh

Acknowledgement

We, the students of B.Tech Computer Science and Engineering (Industrial Internet of Things), 3rd Year, express our sincere gratitude to all those who supported and guided us throughout the development of this project — *Hand Gesture Virtual Mouse System*.

We are deeply thankful to our **Project Guide / Faculty Mentor** for their invaluable guidance, continuous encouragement, and constructive feedback at every stage of the project. Their technical expertise and academic insights were instrumental in shaping this work.

We extend our gratitude to the **Head of the Department** and the **Faculty of the CSE Department** for providing access to laboratory resources, development tools, and an environment conducive to research and innovation.

We also acknowledge the open-source communities behind **MediaPipe**, **OpenCV**, **PyAutoGUI**, and **Flask**, whose freely available tools and documentation made this project possible.

Finally, we thank our families and peers for their unwavering moral support and patience throughout the project duration.

— Kavya, Aarushi, Riya, Shobhit, Tanmay, Vansh, Chitransh, Vivek, Priyanshu, Divya, Mayank

Table of Contents

Acknowledgement	2
Table of Contents	3
Abstract	4
1. Introduction	5
2. Objectives	6
3. Scope of the Project	6
4. Problem Statement	7
5. System Architecture	7
6. Technologies Used	9
7. Functional Requirements	10
8. Non-Functional Requirements	11
9. Implementation Details	11
10. Deployment Strategy	13
11. Challenges Faced	14
12. Solutions Implemented	15
13. Results and Output	15
14. Limitations	16
15. Future Enhancements	17
16. Conclusion	17
17. References	18

Abstract

The rapid convergence of computer vision and artificial intelligence has opened unprecedented opportunities for re-imagining how humans interact with digital systems. This project presents the **Hand Gesture Virtual Mouse System** — a real-time, touchless input solution that replaces the conventional computer mouse with natural hand movements captured by a standard webcam.

The system is built on a modular Python pipeline. **OpenCV** acquires and pre-processes live video frames, while **Google's MediaPipe Hands** model detects and tracks 21 anatomical landmarks on the human hand at near-real-time speeds. A custom gesture-recognition engine analyses the spatial relationships between key landmarks to classify five core gestures: cursor movement, left click, right click, scroll up, and scroll down. The classified gesture is then translated into an operating-system mouse event via **PyAutoGUI**. A lightweight **Flask** web server streams the annotated video feed to a browser-based interface designed with Tailwind CSS.

Experimental evaluation demonstrates accurate gesture detection under standard indoor lighting, stable cursor movement achieved through coordinate smoothing, and reliable click/scroll operations. The system addresses accessibility needs, hygiene-critical environments, and futuristic human-computer interaction scenarios. Future directions include deep-learning-based gesture classifiers, mobile support, and drag-and-drop functionality.

Keywords: Hand Gesture Recognition, Computer Vision, MediaPipe, OpenCV, PyAutoGUI, Flask, Human-Computer Interaction, Touchless Control, Real-Time Processing

1. Introduction

Human-Computer Interaction (HCI) has evolved dramatically over the past five decades — from punch cards and command-line terminals in the 1960s to graphical user interfaces in the 1980s, touchscreens in the 2000s, and now gesture- and voice-based systems. Each paradigm shift has sought to make computing more natural, accessible, and efficient.

The conventional mouse, despite being ubiquitous, imposes physical constraints: it requires contact with a surface, a dedicated workspace, and fine motor control. In contexts such as sterile medical environments, public kiosks, industrial IoT control panels, or assistive technology for users with limb disabilities, a touchless alternative is not merely preferable — it is essential.

This project, **Hand Gesture Virtual Mouse System**, addresses this gap by leveraging state-of-the-art computer vision to interpret hand gestures in real time and map them to standard mouse operations. The core insight is that the human hand is an extraordinarily expressive instrument; even a small vocabulary of discrete gestures is sufficient to replicate all primary mouse functionalities.

The project integrates seamlessly with any desktop environment (Windows, macOS, Linux) and requires no specialised hardware beyond a standard USB or built-in webcam. This makes the solution low-cost, portable, and immediately deployable. A complementary web interface enables remote monitoring and demonstration of the system's operation.

1.1 Motivation

The COVID-19 pandemic underscored the importance of touchless interfaces in shared public spaces. Additionally, the proliferation of IoT and smart environments demands interaction paradigms that extend beyond traditional peripherals. Gesture-based control aligns naturally with the vision of ambient intelligence, where computing is embedded in the environment and controlled through natural human movement.

1.2 Project Context

This project is developed as part of the B.Tech CSE (Industrial Internet of Things) curriculum at the 3rd Year level. It bridges the domains of Artificial Intelligence, Machine Learning, and Embedded/IoT systems by demonstrating how vision-based sensing can replace physical hardware in human-machine interfaces — a core theme in the IIOT paradigm.

2. Objectives

The specific objectives of this project are as follows:

- Develop a **real-time hand gesture recognition pipeline** capable of processing webcam video at a frame rate suitable for responsive interaction (≥ 20 FPS).
- Enable **touchless control of all primary mouse operations** — cursor movement, left click, right click, and scroll — using a small, learnable set of hand gestures.

- Implement **smooth and accurate cursor movement** by applying coordinate interpolation and noise-reduction techniques to raw landmark data.
 - Build a **web-based monitoring interface** using Flask that streams the annotated video feed and displays the currently detected gesture in real time.
 - Demonstrate a **modular, extensible software architecture** that allows future integration of additional gestures, deep-learning classifiers, or IoT device control.
 - Evaluate the system's performance in terms of **gesture detection accuracy, latency, and usability** under typical indoor conditions.
-

3. Scope of the Project

This project is scoped for local deployment on a personal computer and targets single-user interaction scenarios. The following capabilities are within scope:

- Real-time hand detection and landmark tracking via an integrated or USB webcam.
- Gesture-to-action mapping for cursor movement, left/right click, and scrolling.
- A browser-accessible interface for live video monitoring.
- A simplified Gradio-based deployment on Hugging Face Spaces for demonstration purposes (gesture visualisation only; system-level mouse control is not available on cloud platforms).

The following are explicitly out of scope for the current version:

- Multi-user simultaneous gesture tracking.
 - Voice command integration.
 - Mobile device deployment.
 - Drag-and-drop operations (planned for a future version).
-

4. Problem Statement

Traditional input devices — mouse and keyboard — are inherently tactile, requiring the user to physically touch and manipulate hardware. This dependency creates several friction points:

- **Hygiene and infection control:** Shared peripherals in hospitals, kiosks, and labs are potential vectors for pathogen transmission.
- **Accessibility barriers:** Users with motor disabilities, tremors, or limb differences may find conventional mice difficult or impossible to use effectively.
- **Space and mobility constraints:** Operating a mouse requires a flat surface and a relatively fixed posture, limiting interaction flexibility.
- **Integration with smart environments:** IoT and IIOT systems increasingly require interfaces that are embedded in the environment rather than tethered to a desk.

The central problem addressed by this project is: *Can a standard webcam, combined with open-source computer vision libraries, serve as a fully functional replacement for a physical mouse, delivering real-time performance and acceptable usability?* This project answers that question affirmatively and provides a working implementation as evidence.

5. System Architecture

The system is designed around a **five-layer modular pipeline**. Each layer has a single, well-defined responsibility, making the system easy to test, maintain, and extend. Data flows sequentially from the input layer to the action layer, with the interface layer operating as a parallel output stream.

5.1 Layer-by-Layer Description

Layer 1 – Input	Webcam captures live video frames at ~30 FPS using OpenCV's VideoCapture API. Each frame is a 3-channel BGR image that is subsequently converted to RGB for MediaPipe processing.
Layer 2 – Processing	MediaPipe Hands processes every RGB frame and returns a set of 21 3-D landmarks per detected hand, each normalised to [0, 1] relative to the frame bounding box.
Layer 3 – Gesture Engine	Custom Python logic computes Euclidean distances and angular relationships between landmark pairs (e.g., thumb-tip ↔ index-tip) to classify gestures with configurable threshold values.
Layer 4 – Action	PyAutoGUI translates the classified gestures into OS-level mouse events: moveTo(), click(), rightClick(), and scroll(). Coordinate mapping uses linear interpolation from camera space to screen space.
Layer 5 – Interface	A Flask web server streams the annotated video feed via MJPEG over HTTP. An HTML/Tailwind CSS front-end renders the stream alongside a live gesture-status panel.

Figure 1: Five-Layer System Architecture of the Hand Gesture Virtual Mouse System

5.2 Data Flow Diagram (Textual Representation)

The following describes the end-to-end data flow through the system:

Step	Component	Input → Output
1	OpenCV	Webcam stream → RGB frame (640×480)
2	MediaPipe	RGB frame → 21 (x, y, z) landmarks
3	Gesture Engine	Landmarks → Gesture label (e.g., LEFT_CLICK)
4	PyAutoGUI	Gesture label → OS mouse event
5	Flask + Browser	Annotated frame → MJPEG stream displayed in browser

6. Technologies Used

The project leverages a carefully selected stack of open-source Python libraries and web technologies. Each component was chosen for its maturity, performance, and suitability for real-time computer vision tasks.

Technology	Version / Tool	Role in System
Python	3.9+	Core programming language for all modules
OpenCV	4.x	Real-time video capture and frame processing
MediaPipe	0.10.x	Hand landmark detection (21 key points per hand)
PyAutoGUI	0.9.x	Cross-platform mouse and keyboard automation
Flask	2.x	Lightweight web server for the UI interface
HTML + Tailwind CSS	CDN	Responsive front-end design and layout
NumPy	1.x	Numerical computations for coordinate mapping

Table 1: Technology Stack Summary

6.1 MediaPipe Hands — Core AI Component

MediaPipe Hands is a machine-learning pipeline developed by Google that uses a two-stage detection approach: a palm-detection model identifies the approximate hand region, and a hand-landmark model precisely locates 21 key points in 3-D space. The model runs in real time on CPU, making it well-suited for deployment without specialised hardware (e.g., GPU or neural processing unit). The 21 landmarks correspond to anatomical joints: wrist, MCP, PIP, DIP, and tip for each of the five fingers.

6.2 OpenCV — Vision Pipeline

OpenCV (Open Source Computer Vision Library) manages the video capture loop, colour space conversion (BGR to RGB required by MediaPipe), frame annotation (drawing landmarks and gesture labels), and JPEG encoding for the web stream. Its highly optimised C++ backend ensures minimal per-frame processing overhead.

6.3 PyAutoGUI — OS Integration

PyAutoGUI provides a cross-platform Python interface to OS-level GUI automation. It translates high-level commands such as `moveTo(x, y)`, `click()`, `rightClick()`, and `scroll(units)` into native mouse events via the platform's accessibility APIs. A configurable FAILSAFE

mechanism (moving the cursor to a corner aborts execution) prevents runaway automation.

7. Functional Requirements

Functional requirements define the specific behaviours the system must exhibit. Each requirement is described with its expected input, processing logic, and output.

ID	Requirement	Description
FR-01	Hand Detection	The system shall detect and track at least one hand in the live webcam feed at a minimum of 20 FPS under standard indoor illumination (≥ 200 lux).
FR-02	Landmark Extraction	Upon detecting a hand, the system shall extract the (x, y) normalised coordinates of all 21 MediaPipe landmarks and convert them to pixel coordinates relative to the frame dimensions.
FR-03	Cursor Movement	The vertical and horizontal position of the index finger tip (Landmark 8) shall be linearly mapped to the full screen resolution and applied to move the system cursor.
FR-04	Left Click	When the Euclidean distance between the thumb tip (Landmark 4) and index finger tip (Landmark 8) falls below a configurable threshold (default: 40 pixels), a single left mouse click shall be triggered.
FR-05	Right Click	When the Euclidean distance between the thumb tip (Landmark 4) and middle finger tip (Landmark 12) falls below the threshold, a single right mouse click shall be triggered.
FR-06	Scroll	A vertical displacement of both the index (Landmark 8) and middle finger (Landmark 12) tips exceeding 20 pixels per frame shall trigger a scroll-up or scroll-down event proportional to displacement magnitude.
FR-07	Web Interface	A Flask server shall stream the annotated camera feed via MJPEG to a browser-accessible URL (localhost:5000) and display the currently recognised gesture label.

8. Non-Functional Requirements

Attribute	Requirement
Performance	End-to-end latency from gesture to mouse event shall not exceed 150 ms under normal operating conditions.
Accuracy	Gesture classification accuracy shall exceed 90% in a controlled lighting environment with a plain background.
Smoothness	Cursor movement shall be smooth, with jitter not perceptible to a typical user; achieved via exponential moving average smoothing.
Usability	A first-time user shall be able to learn and use the five core gestures within five minutes without any written instructions.
Reliability	The system shall operate continuously for a minimum of one hour without crashes, memory leaks, or significant performance degradation.
Portability	The system shall run on Windows 10+, macOS 12+, and Ubuntu 20.04+ without code modification.
Maintainability	The codebase shall be organised into modules (capture, detect, gesture, control, server) with inline documentation.

9. Implementation Details

This section provides a detailed account of the key implementation decisions, algorithms, and code-level techniques employed in the system.

9.1 Hand Landmark Model

MediaPipe Hands detects 21 three-dimensional landmarks per hand. Each landmark has an index from 0 (wrist) to 20 (pinky tip). The landmarks of primary interest for this system are:

Landmark ID	Anatomical Point	Role in System
4	Thumb Tip	Click detection anchor
8	Index Finger Tip	Cursor position + left click
12	Middle Finger Tip	Right click + scroll partner
0	Wrist	Reference for relative calculations

Table 2: Key MediaPipe Hand Landmarks

9.2 Gesture Recognition Algorithm

Gesture detection is based on computing the **Euclidean distance** between pairs of landmark pixel coordinates:

$$\text{distance}(\mathbf{A}, \mathbf{B}) = \sqrt{[(x_A - x_B)^2 + (y_A - y_B)^2]}$$

A distance below the configured threshold triggers the corresponding action. To prevent false positives from transient noise, a gesture must be sustained for at least two consecutive frames before the action is executed. A **cooldown timer** (default: 500 ms) prevents accidental repeated clicks.

Gesture	Landmarks Involved	Distance Threshold	Mouse Action
Index finger up	Landmark 8 (index tip)	Vertical position	Cursor movement
Thumb + Index pinch	Landmarks 4 & 8	< 40 px	Left click
Thumb + Middle pinch	Landmarks 4 & 12	< 40 px	Right click
Index + Middle swipe	Landmarks 8 & 12 (Y-axis delta)	> 20 px/frame	Scroll up / down
All fingers closed (fist)	All fingertips close to palm	< 30 px	Pause / no action

Table 3: Gesture-to-Action Mapping

9.3 Coordinate Mapping and Smoothing

Raw landmark coordinates are normalised to [0, 1]. To obtain screen pixel coordinates:

- The normalised x and y values of Landmark 8 are multiplied by the screen width and height respectively (obtained via `PyAutoGUI.size()`).
- An **Exponential Moving Average (EMA)** filter is applied: $\text{smooth_x} = \alpha \times \text{raw_x} + (1 - \alpha) \times \text{prev_x}$, where $\alpha = 0.5$. This significantly reduces cursor jitter without introducing noticeable lag.
- The mapped coordinates are clamped to the screen boundaries to prevent out-of-range errors.

9.4 Flask Web Interface

The Flask server runs in a separate thread from the gesture-recognition loop. It exposes two routes: `/` (serves the HTML front-end) and `/video_feed` (streams MJPEG frames). Each annotated frame is JPEG-encoded by OpenCV, wrapped in a multipart HTTP response, and pushed to the browser. The HTML interface uses Tailwind CSS utility classes for a clean, responsive layout and displays the current gesture label fetched from a shared Python variable.

10. Deployment Strategy

10.1 Local Deployment

The full system runs locally and requires the following setup steps:

- Install Python 3.9+ and create a virtual environment.
- Install dependencies: `pip install opencv-python mediapipe pyautogui flask numpy`.
- Run the main application: `python app.py`.
- Open a browser and navigate to `http://localhost:5000` to view the live feed.
- Position the hand within the webcam frame; gestures are recognised immediately.

Local deployment supports the complete feature set, including real-time cursor control. The system has been tested on Windows 10, Ubuntu 22.04, and macOS 13 (Ventura).

10.2 Cloud Deployment — Hugging Face Spaces

Cloud platforms present fundamental constraints for this application:

- Sandboxed environments do not permit access to the host operating system's mouse/keyboard APIs; PyAutoGUI calls are therefore non-functional.
- Webcam access from a remote browser via WebRTC introduces latency unsuitable for real-time control.

To address these constraints, a simplified **Gradio-based demonstration application** is deployed on Hugging Face Spaces. This version accepts a static image or short video clip as input, processes it through the MediaPipe pipeline, and returns an annotated image showing detected landmarks and the inferred gesture label. While it cannot control the mouse, it effectively demonstrates the gesture-recognition core of the system to remote evaluators.

10.3 Deployment Comparison

Feature	Local (Flask)	Cloud (Hugging Face)
Real-time webcam	✓ Supported	✗ Not supported
Mouse control	✓ Full control	✗ Sandboxed
Gesture visualisation	✓ Supported	✓ Supported
Public accessibility	✗ Local only	✓ Public URL
Latency	< 50 ms	Variable (network)

11. Challenges Faced

Gesture Ambiguity

Several gesture combinations produce landmark configurations that are visually similar. For example, a relaxed index finger and a partially closed fist can produce landmark distances close to the left-click threshold, resulting in unintended clicks during cursor movement.

Cursor Jitter

Raw landmark coordinates from MediaPipe contain per-frame noise on the order of 2–5 pixels. Without smoothing, this noise manifests as visible cursor jitter, degrading usability significantly.

Lighting Sensitivity

MediaPipe's hand detection confidence drops in low-light conditions (< 100 lux) and in high-contrast or backlit environments. This caused intermittent tracking failures during initial testing.

Cloud Platform Restrictions

Deploying the full system to a cloud platform proved impossible due to OS-level sandboxing preventing PyAutoGUI from executing mouse events, and the absence of real-time webcam access from the server side.

MediaPipe Version Compatibility

Breaking API changes between MediaPipe 0.9.x and 0.10.x required code refactoring. Specifically, the `hands.process()` return signature changed, causing `AttributeErrors` that were initially difficult to diagnose.

Click Repeat Prevention

Without a cooldown mechanism, a sustained pinch gesture triggered dozens of clicks per second, making single-click operations unreliable.

12. Solutions Implemented

Unique Gesture Mapping

Each gesture was assigned a distinct, non-overlapping landmark combination. A minimum angular separation and distance margin between gesture thresholds was enforced to reduce false positive rates.

Exponential Moving Average (EMA) Smoothing

An EMA filter with $\alpha = 0.5$ was applied to both x and y cursor coordinates, effectively averaging noise across frames while preserving responsiveness to intentional movement.

Adaptive Lighting Guidance

The interface displays a real-time confidence score from MediaPipe. If confidence drops below 0.75, an on-screen warning prompts the user to improve lighting. Pre-processing (histogram equalisation) was explored but found to introduce latency.

Separate Cloud Deployment Strategy

A Gradio application was developed independently for cloud deployment, with a clear separation between the demonstration layer (gesture visualisation) and the control layer (mouse automation).

Versioned Dependency Management

A requirements.txt file pins all library versions to ensure reproducibility. Compatibility was verified for Python 3.9 and 3.10.

Click Cooldown Timer

A 500 ms cooldown timer was implemented using Python's time.time(). After a click event, no further click is registered until the timer expires, enabling reliable single-click and double-click detection.

13. Results and Output

The system was evaluated through structured testing sessions involving five student users performing predefined tasks (cursor navigation, text selection, web browsing) using only hand gestures. The key outcomes are summarised below.

Metric	Observed Value	Notes
Hand Detection FPS	24–28 FPS	On a mid-range laptop, CPU-only
End-to-End Latency	80–120 ms	Gesture to OS mouse event
Gesture Accuracy	~93%	Good lighting, plain background
Cursor Smoothness	Satisfactory	EMA smoothing applied ($\alpha=0.5$)
Left Click Success Rate	96%	With 500 ms cooldown
Right Click Success Rate	94%	With 500 ms cooldown
Scroll Reliability	91%	Vertical displacement > 20 px/frame
User Learning Time	< 5 minutes	All 5 test users

Table 4: Performance Evaluation Results

13.1 Key Observations

- The system maintains real-time performance without a GPU, making it accessible on commodity hardware.
- Gesture recognition accuracy degrades gracefully in sub-optimal conditions rather than failing abruptly.
- Users with no prior experience with gesture-based interfaces achieved functional proficiency within minutes.

- The Flask web interface successfully streamed the annotated feed at 24 FPS with no observable browser-side lag on a local network.
-

14. Limitations

While the system achieves its primary objectives, several limitations have been identified that scope its applicability and should be addressed in future iterations:

Lighting Dependency

The system requires adequate, uniform illumination. In low-light environments or against high-contrast backgrounds, MediaPipe's detection confidence drops below the functional threshold, causing tracking interruptions.

Single-User Constraint

The current implementation processes only one hand at a time. In environments where multiple people are in the camera's field of view, the system may track the wrong hand.

No Cloud Mouse Control

System-level mouse automation cannot be replicated in sandboxed cloud environments. The cloud deployment is limited to gesture visualisation only.

Fixed Gesture Vocabulary

The system supports exactly five gestures. Users requiring additional operations (e.g., double-click, drag, keyboard shortcuts) cannot perform them without a physical device as a supplement.

Complex Background Interference

Highly textured or cluttered backgrounds can occasionally cause false hand detections, particularly at the edges of the camera frame.

Fatigue Over Extended Use

Holding the hand elevated in front of a webcam for extended periods can cause arm fatigue. The system is not recommended as a full-time mouse replacement without ergonomic support.

15. Future Enhancements

The current system establishes a solid foundation for touchless interaction. The following enhancements are proposed for future development cycles:

- Deep Learning Gesture Classifier: Replace the threshold-based engine with a trained Convolutional Neural Network (CNN) or LSTM model to support a richer gesture vocabulary and improve accuracy in challenging conditions.
- Drag and Drop: Implement a sustained pinch-and-move gesture for drag-and-drop operations, enabling file management tasks via gesture.

- **Gesture Customisation Interface:** Provide a GUI that allows users to record and assign custom gestures to arbitrary keyboard shortcuts or application commands.
 - **Browser-Native Version:** Use WebAssembly (WASM) to port the MediaPipe pipeline to the browser, enabling a fully client-side solution without a Python backend.
 - **Mobile Device Support:** Adapt the system for smartphone/tablet cameras using TensorFlow Lite and the Android/iOS MediaPipe SDKs.
 - **Two-Hand Interaction:** Extend the pipeline to simultaneously track both hands, enabling richer bimanual gesture vocabularies (e.g., zoom via two-finger pinch).
 - **IoT Integration:** Connect the gesture recognition output to MQTT brokers or REST APIs to control smart home devices, industrial actuators, or robotic arms — directly aligning with IIOT applications.
-

16. Conclusion

This project successfully demonstrates that a touchless, gesture-driven virtual mouse can be implemented using off-the-shelf, open-source tools at no additional hardware cost beyond a standard webcam. The **Hand Gesture Virtual Mouse System** achieves real-time hand tracking, accurate gesture classification, smooth cursor control, and reliable click and scroll operations — meeting all primary functional and non-functional requirements.

From an academic perspective, this project integrates concepts from multiple domains: computer vision (image acquisition, feature extraction), machine learning (pre-trained landmark detection models), software engineering (modular architecture, web integration), and human-computer interaction (usability, latency, ergonomics). This interdisciplinary integration is emblematic of the IIOT domain, where physical sensing, intelligent processing, and digital actuation converge.

The challenges encountered — gesture ambiguity, cursor jitter, cloud deployment constraints, and library compatibility issues — were systematically addressed through engineering solutions including EMA smoothing, cooldown timers, unique gesture mapping, and version-pinned dependency management. These problem-solving experiences constitute valuable learning outcomes beyond the technical implementation itself.

Looking ahead, the addition of deep-learning classifiers, expanded gesture vocabularies, browser-native deployment, and IoT device control will substantially increase the system's practical utility. The project thus serves both as a functional prototype and as a roadmap for future innovation in touchless human-machine interfaces.

17. References

[1] Zhang, F., et al. (2020). *MediaPipe Hands: On-device Real-time Hand Tracking*. Workshop on Computer Vision for Augmented and Virtual Reality, ECCV 2020. Available at: <https://arxiv.org/abs/2006.10214>

- [2] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. Available at: <https://opencv.org/>
- [3] Al Sweigart. (2019). *Automate the Boring Stuff with Python*, 2nd ed. No Starch Press. (PyAutoGUI documentation: <https://pyautogui.readthedocs.io/>)
- [4] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*, 2nd ed. O'Reilly Media. (Flask documentation: <https://flask.palletsprojects.com/>)
- [5] Google LLC. (2023). *MediaPipe Solutions Guide — Hands*. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
- [6] OpenCV Development Team. (2023). *OpenCV 4.x Documentation*. <https://docs.opencv.org/4.x/>
- [7] Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace. (<https://docs.python.org/3/>)
- [8] Mitra, S., & Acharya, T. (2007). Gesture Recognition: A Survey. *IEEE Transactions on Systems, Man, and Cybernetics*, 37(3), 311–324.
- [9] Rautaray, S. S., & Agrawal, A. (2015). Vision based hand gesture recognition for human-computer interaction: a survey. *Artificial Intelligence Review*, 43(1), 1–54.
- [10] Tailwind CSS. (2023). *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>